

LocoTasks

CMPSC 475 Final Project Proposal

Micah DeVore

April 8th, 2026

Table of Contents

1 Introduction.....	3
1.1 App Definition Statement.....	3
2 Features	3
2.1 Track User’s Location	3
2.2 Add Tasks (First Method)	3
2.3 Persistence of Tasks	4
2.4 Add Tasks based on Map Pin (Second Method)	4
2.5 Task Annotations.....	4
2.6 Task List View	4
2.7 Add Emotions.....	4
2.8 Create Emotion Graph.....	5
2.9 Create/Join a “Friend Group”	5
2.10 Add Tasks to other Users' Accounts	5
2.11 Task Management Game.....	5
3 User Experience	6
3.1 Login + Sign Up Pages.....	6
3.2 Home Page	6
3.3 List Views	7
3.4 Map View	7
3.5 Account View.....	8
4 Technical Notes	8
4.1 Backend.....	8
4.2 Frontend	9
5 Schedule.....	9

1 Introduction

1.1 App Definition Statement

LocoTasks is a multi-purpose app designed to accomplish two main functions: task management and journal entries based on user location. This app is great for people who need a reminder for something that is not based on time but instead position. The purpose of this app is to allow users to create tasks for themselves or others in a fun way that pop up based on the user's location. Additionally, users can record journal entries and emotions, allowing them to see which locations cause which emotions. This app will have four essential features: task manager based on location, ability to add tasks to another user's app, tracking user's emotions based on location, and a task manager game. LocoTasks stands out by making task management fun while also incorporating cooperation between users.

2 Features

2.1 Track User's Location

Description: Although simple, this is a core feature of the app as everything is built off being able to track the user's location.

Implementation Details: Use Apple's built in MapKit and delegates to easily keep track of user's location based on where their IOS is. This step includes creating a map view for the app. Additionally, the app should display a region around the user's location.

2.2 Add Tasks (First Method)

Description: User will have the ability to add a task by inputting text into a title field and description field (Both are required fields). Users can also assign a number to each task for its difficulty (which is used later for scoring), as well as add latitude and longitude coordinates (current location or specified) to attach the task to a location. Both of these are optional. Users can also click a button that signals if the task has been completed or not.

Implementation Details: Create a swift view that shows all current tasks the user has created. Additionally, add a button that brings up a sheet view which allows the user to create a new task by inputting title and description. Add a field for difficulty. Add two more fields for latitude and longitude coordinates which will allow a user to attach the task to a location. Thus, this task will show up for that location when a button is clicked (future feature). Additionally, the user will have the option to autofill the latitude and longitude coordinates based on their current location. This allows the user to add tasks based on their current location fast and easily.

2.3 Persistence of Tasks

Description: Allow tasks to persist even when a user logs out.

Implementation Details: Create a backend server that contains a database of tasks. This will keep track of user's accounts, user's tasks, and any other needed information for future features. Due to the scope of this project, only a local server will be implemented.

2.4 Add Tasks based on Map Pin (Second Method)

Description: Users will have the ability to click on the map and then add a task to that specific location.

Implementation Details: To implement this, users will first drop a pin on the map. Then, based off this pin, users will have the ability to click an "Add Task" button that will autofill the coordinates for this location. Users will still have the option to add their own title, description, and difficulty to the task.

2.5 Task Annotations

Description: Users can click a button that shows all tasks displayed on the map by location.

Implementation Details: When the button is clicked, an annotation for each task is created and displayed on the map based on the coordinates given for each task. When the button is clicked again, all the annotations are cleared. This allows the user to see where the majority of their tasks are located.

2.6 Task List View

Description: Users can navigate to a view that lists all current tasks, all completed tasks, and all tasks within a specific range of their location.

Implementation Details: Use a picker to create three separate lists. Additionally, implement search functionality so users can search for a specific task. Use coordinates to check if task is located with a specific range of current users' location.

2.7 Add Emotions

Description: Users will have the ability to add emotions based off their current location and then display those emotions on the map using a button.

Implementation Details: Repeat features 2.2 to 2.6, but instead of tasks, do it for emotions. The only difference is that users will implement a journal entry, location, and the emotion they are currently feeling. This emotion will be picked from a list so that we can keep track of the emotions on the backend for the next feature.

2.8 Create Emotion Graph

Description: Users will have the ability to view a Pie graph of all emotions or only emotions within a specific range of their location.

Implementation Details: Calculate a total of each emotion from the journal entries that are within a specific radius. Also calculate the total for each emotion. Then, display two pie graphs. Use a picker to choose which pie graph to show. NOTE: If finding them within a specific location is too difficult, then just display the total of each emotion for ALL locations.

2.9 Create/Join a “Friend Group”

Description: Users will have the ability to create a “Friend Group” which allows them to invite friends. This will then allow users to execute the next few features.

Implementation Details: When a user creates a “Friend Group”, the user will give it a name, and the backend server will give this group a unique id. Once created and stored, the server will then add the username to the list of people within the group. Next, the author will have the ability to invite friends to join the group. For simplicity, the author must already know the username of the friend they want to invite. Once they invite the friend, this will send a request to the backend which will populate a view on the friend’s side (so they can see they have a request). Lastly, if the friend accepts, their username will be added to the group. A view will be created to see each group the user is in, and all the members within each group.

2.10 Add Tasks to other Users' Accounts

Description: Users will have the ability to add tasks for other users within their “Friend Group.”

Implementation Details: Once in a group, a button will appear next to each person's name where the current user can click. This button will reuse the create task view, but instead of creating the task for themselves, it will send an API request to populate the friend’s database. Thus, it will add the task to the friend’s account, and not the current user.

2.11 Task Management Game

Description: Users will have the ability to see a screen showing the “scores” of each member of the group. This will allow friendly competition between group members.

Implementation Details: Earlier, it was mentioned that each task gets assigned a number for the difficulty of the task. When a user completes a task, this number gets added to their total points. Thus, a swift view will be created that shows the total points of each member of the group and shows a leaderboard. Due to the scope of this project, no mechanisms to prevent cheating will be created. This game between friends will be based purely off the honors system.

3 User Experience

3.1 Login + Sign Up Pages

Description: Figure 1 shows the initial login page that the user will experience. Additionally, shown is a sign-up page where new users can create an account. Passwords must be at least 8 characters long.

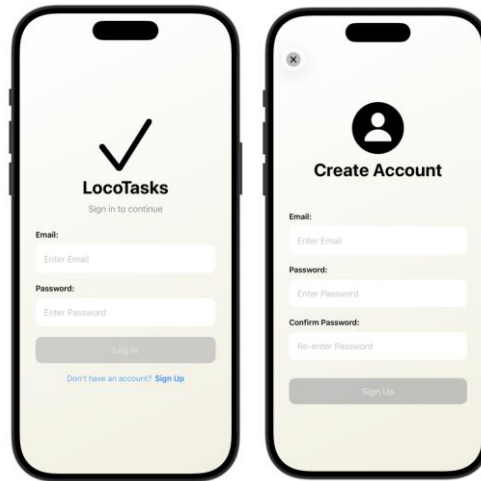


Figure 1: Swift UI mock-ups for login and sign-up pages.

3.2 Home Page

Description: After logging in, users will be shown a home page. Figure 2 shows an example home page where the user can navigate to the map view, emotion list view, task list view, groups list view, and account view. Note that this image is AI generated using Gemini, and the actual design may differ.

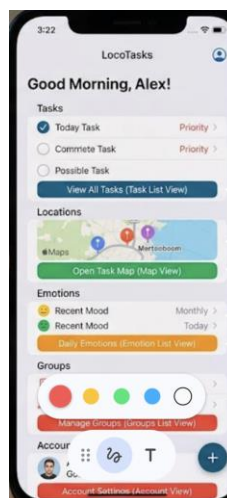


Figure 2: AI generated image of an example home page.

3.3 List Views

Description: Figure 3 shows an example of what the emotion list view, task list view, and group list view will look like. Obviously, each will have its own view respectively. Additionally, the leaderboard for the task management game will be a tab in the group list view.

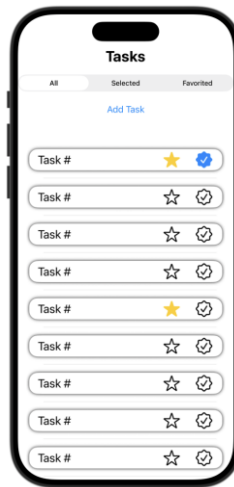


Figure 3: Swift UI example Task List View.

3.4 Map View

Description: Figure 4 shows an example map view with annotations. Our annotations will be with emotions and tasks, adding a clear way to distinguish between them. Notice the buttons that toggle emotion annotations and task annotations.

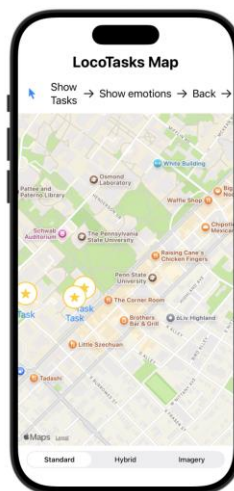


Figure 4: Swift UI example Map View with Task Annotations showing.

3.5 Account View

Description: Figure 5 shows an example account view where the user can logout. This is where users could also potentially change account details. This would only be implemented if expanded on this app. However, due to the scope of this project, changing account details will not be implemented.

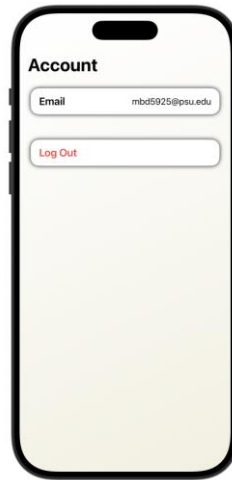


Figure 5: Swift UI example Account View.

4 Technical Notes

4.1 Backend

Description: The backend for this project will use the given TasklyAPI files as starter code. Thus, this means using FastAPI and pickleDB. Then, the given files will be adjusted to fit the needed frontend support such as “Friend Groups” and logging emotions.

4.2 Frontend

Description: The frontend of the application will be using IOS, Swift, Swift UI, and MapKit. These will be used to create the views for user interaction with the backend.

5 Schedule

■ Week 1: 4/12 - 4/18

This week will focus on the setup of the app and creating the main swift views. Because this app is very similar to our previous apps, Taskly and Campus App, I will pull views and code from these two apps to help start the structure of my LocoTasks App. Thus, the goal of this week is to set up the new app and implement views from both apps. No backend logic is included in this. Only finishing all the views.

- ☐ Create LocoTasks Project in XCode.
- ☐ Pull all needed views, backend, etc. from Taskly and Campus App.
- ☐ Merge views to work for LocoTasks. Additionally, create new views to represent emotions, task manager game, pie chart, and anything else needed.

■ Week 2: 4/19 - 4/25

This week will focus on Features 2.1 to 2.8. Because the views should already be created, this week will focus on finishing the backend logic for these features. The goal of this week is to get the full functionality of these features working.

- ☐ Create tasks and emotions in the backend, then connect to views.
- ☐ Implement adding tasks and emotions based on current location or dropped pin.
- ☐ Test functionality of all features to make sure correct.

■ Week 3: 4/26 - 5/2

This week will focus on implementing Features 2.9 to 2.11. I have set aside a whole week for these features because features 2.9 and 2.10 will be a challenging task to finish and test. Thus, the goal is to get these working and allow users to connect to a “Friend Group” and add tasks to each other's account.

- ☐ Create “Friend Groups” on the backend.
- ☐ Create ability to join groups and add tasks to other accounts.
- ☐ Test to make sure all functionality is working.
- ☐ Implement task management game (leaderboard).

■ Week 4: 5/3 - 5/4

The goal is to finish everything before this week. However, this week will be used for any issues that come up in the previous weeks. Thus, this will be a catch-up and final-touches week.

- ☐ Add final touches.
- ☐ Fix final issues
- ☐ Conduct final testing